

PROJECT PROPOSAL

Book Store

An Online Book Inventory, Discovery, and Member Platform

A full-stack web application for browsing, saving, and managing books,
with role-based access for readers and administrators.

Frontend: React 18, Vite, Tailwind CSS, React Router

Backend: FastAPI (Python)

Data: PostgreSQL (authentication) · MongoDB (books & reviews)

DOCUMENT DETAILS

Version 1.0 · August 2026

Prepared from the current Book Store codebase

Local client: <http://localhost:5173> API: <http://127.0.0.1:5000>

1. Title Page Details

This proposal describes the **Book Store** web system currently implemented in the project repository. Blank fields below may be filled for academic submission.

Field	Value
Project title	Book Store — Online Book Inventory and Reader Platform
Project type	Full-stack web application (academic / portfolio)
Submitted by	_____
Roll / ID	_____
Supervisor	_____
Institution	_____
Department	Computer Science / Software Engineering
Date	August 2026
Document version	1.0

2. Abstract

Book Store is a web-based book inventory and discovery platform. Readers can browse a catalog, search and filter by title, author, or genre, open full book pages, and follow real purchase or listing links on Amazon and Goodreads. Registered members can save books to a personal list and post reviews. An administrator manages the catalog through a protected dashboard (upload, edit, delete) and can view inventory statistics. Authentication is implemented on the FastAPI backend with PostgreSQL (bcrypt password hashing and JWT sessions). Book records and reviews are stored in MongoDB. The client is a React single-page application built with Vite, Tailwind CSS, and React Router. This proposal presents the problem, objectives, architecture, modules, data design, APIs, and future work based on the implemented system.

3. Introduction and Problem Statement

Many small bookshops and student projects still rely on static listings or fake checkout flows. Readers cannot see complete metadata, cannot keep a personal list, and cannot trust that “Buy” actually leads to a real store. Administrators often lack a simple inventory panel. The Book Store project addresses these gaps with a single web application that separates public browsing from privileged catalog management.

3.1 Problem

- Catalog data (title, author, cover, genre, rating, year) is often incomplete on simple listing sites.
- Guests and members need different capabilities: browse vs. save, review, and manage stock.
- Authentication must be owned by the project (not a third-party auth SaaS) and stored in PostgreSQL.
- Administrators need a dedicated dashboard that ordinary users cannot open.

3.2 Proposed solution

The system is a two-tier web application: a React client and a FastAPI REST API. Public pages (Home, Shop, About, Contact, book details) work without login. Membership unlocks reviews and saved books. Only an account with role **admin** can open **/admin/dashboard**. Passwords never leave the server in plain text; the browser stores a JWT after login or sign-up so the user is signed in immediately after registration.

4. Objectives

4.1 General objective

To design and implement an online book store that supports discovery, membership features, and administrative inventory control using a modern JavaScript frontend and a Python API with relational and document databases.

4.2 Specific objectives

- Provide a responsive public catalog with search and genre filters.
- Show complete book details and outbound links to Amazon search and original listings (e.g. Goodreads).
- Register and authenticate users in PostgreSQL with hashed passwords and JWT.
- Allow members to save books and submit star-rated reviews.
- Restrict catalog create/update/delete and dashboard analytics to administrators.
- Keep books and reviews in MongoDB for flexible document storage.
- Expose a documented REST API (FastAPI interactive docs on port 5000).

5. Scope

5.1 In scope

- Public marketing and catalog pages: Home, About, Shop, Contact, book detail.
- Member account: sign up, login, logout, saved books, reviews.
- Admin dashboard: stats, upload book, manage/edit/delete books.
- REST API for books, reviews, register, login, and current user (/me).
- Local development on Windows (Vite :5173, FastAPI :5000).

5.2 Out of scope (current version)

- In-app payment gateway and shopping cart checkout (Buy opens Amazon).
- Email-based password reset and Google/OAuth social login.
- Mobile native apps (the UI is responsive in the browser).
- Migrating the entire book catalog from MongoDB into PostgreSQL.
- Production hosting, HTTPS termination, and rate limiting at scale.

6. Target Users and Roles

Role	Who	What they can do
Guest	Visitor without an account	Home, About, Shop, Contact, book pages. Cannot save books or post reviews.

Role	Who	What they can do
User (member)	Registered reader	All guest features plus Sign up/Login (JWT), Save book, Account list, write reviews.
Admin	Seeded store manager (role = admin)	All member features plus Dashboard, Upload, Manage, Edit, Delete inventory.

Table 1. Role-based access. Admin routes are wrapped in a private router that requires a logged-in user whose role is admin; other users are sent home.

7. System Architecture

The architecture is a browser SPA talking to a single REST backend. The API talks to two data stores: PostgreSQL for identity, MongoDB for catalog content.

Layer	Component	Responsibility
Presentation	React 18 + Vite + Tailwind + Flowbite	Pages, forms, routing, JWT in localStorage, saved-book list on the device.
API	FastAPI + Uvicorn (port 5000)	CORS-enabled REST: books, reviews, register, login, /me. JWT Bearer on protected routes.
Identity DB	PostgreSQL (database bookstore, table users)	Email (unique), bcrypt hash, role, display name, created_at. Admin seeded on startup.
Catalog DB	MongoDB (database Bookinventory)	Collection books (inventory). Collection reviews (reader comments, ratings).
External	Amazon search URL, Goodreads/listing URL	Purchase and reference; not processed inside this application.

Table 2. Logical architecture of Book Store.

Request flow (member review): Browser → POST /add-review with Authorization: Bearer <token> → FastAPI validates JWT → loads user from PostgreSQL → writes review document to MongoDB → JSON success → Home review carousel refreshes.

8. Methodology

Development followed an iterative, module-based approach similar to Agile: public catalog first, then inventory API, then authentication and membership features, then admin dashboard hardening. Each slice was tested in the browser against the local API.

Phase	Focus	Outcome
1. Requirements	Reader vs admin needs	Pages, roles, and data fields defined
2. Catalog API	FastAPI + MongoDB books	CRUD matching the React forms
3. Public UI	Home, Shop, About, book page	Search, filters, Amazon/Goodreads
4. Auth	PostgreSQL users + JWT	Register, login, /me, hashed passwords
5. Membership	Reviews, saved books, Account	Guests blocked from those actions

Phase	Focus	Outcome
6. Admin	Private dashboard	Stats, upload, manage, edit

Table 3. Development phases.

9. Functional Requirements (Modules)

9.1 Public website

- **Home:** hero banner, best-seller carousel, favourite-book section, promo, other titles, live reviews.
- **Shop:** loads GET /all-books; search by title/author/genre; genre chips; cards link to /book/:id.
- **Book detail:** cover, title, author, genre, rating, year; Buy on Amazon; Open on Goodreads (from stored URL); Save if logged in.
- **About:** store story, values, and calls to shop or contact.
- **Contact:** name, email, and message form (client-side submission in the current build).

9.2 Authentication and account

- **Sign up:** POST /register — creates PostgreSQL user with role user, returns JWT, redirects (user is logged in immediately).
- **Login:** POST /login — verifies bcrypt hash, returns JWT and role; admin is sent to the dashboard.
- **Session:** token in localStorage; GET /me restores the session on reload.
- **Account:** profile summary and saved-book grid (stored per email in the browser).
- The reserved admin email cannot be used for public sign-up.

9.3 Reviews

GET /all-reviews is public. POST /add-review requires a valid JWT. The server sets the reviewer name from the logged-in account. Guests see Sign in / Create account instead of the form. Ratings are 1–5 stars.

9.4 Admin dashboard

- Protected layout: PrivateRouter checks login and isAdmin.
- Dashboard home: book count, genre count, review count, average rating, recent books and reviews.
- Upload book: title, author, image URL, listing URL, rating, year, category (Fiction, Programming, History, etc.).
- Manage / Edit: update or delete existing MongoDB book documents.

10. Non-Functional Requirements

- **Security:** bcrypt hashes; JWT (HS256, seven-day expiry); admin routes blocked in the UI; review POST requires Bearer token.
- **Usability:** Tailwind layout, sticky navbar, mobile menu, consistent blue/teal theme.
- **Performance:** Vite HMR for development; async FastAPI (Motor + SQLAlchemy async).
- **Maintainability:** Pydantic models for payloads; environment variables for Mongo URI, DATABASE_URL, JWT secret, admin seed.
- **Portability:** runs locally on Windows with Python 3, Node.js, PostgreSQL, and MongoDB.

11. Technology Stack

Layer	Technology	Why it is used
UI library	React 18	Component pages, context for auth state
Bundler	Vite 5	Fast local dev server and production build
Styling	Tailwind CSS 3, Flowbite React	Utility layout and form controls
Routing	React Router 6	Public routes, loaders for book pages, nested admin routes
Carousel	Swiper	Home book and review sliders
API	FastAPI, Uvicorn	Typed REST, OpenAPI docs at /docs
Validation	Pydantic	Book, review, and auth request bodies
Auth crypto	bcrypt, PyJWT	Password hashing and session tokens
ORM / PG	SQLAlchemy 2, asyncpg	Async PostgreSQL users table
Documents	Motor / PyMongo	Async MongoDB for books and reviews
Config	python-dotenv	Local .env for secrets and URLs

Table 4. Tools and libraries used in the repository.

12. Data Design

12.1 PostgreSQL — users

Column	Type	Notes
id	INTEGER PK	Auto-increment
email	VARCHAR(120) UNIQUE	Lowercased login key
password_hash	VARCHAR(255)	bcrypt hash, never returned to the client
role	VARCHAR(20)	user or admin
display_name	VARCHAR(80)	Defaults to the email local-part
created_at	TIMESTAMPTZ	Account creation time

Table 5. Identity schema (bookstore.users).

12.2 MongoDB — books

Each book document includes an ObjectId **_id** plus fields used by upload and shop screens: **title**, **author**, **imgURL**, **bookpdf** (external listing URL), **rating**, **publishedYear**, **genre**. Extra keys are allowed on insert so older records still load.

12.3 MongoDB — reviews

Review documents store **name** (from the authenticated user), **role** (optional city or label), **comment**, **rating** (1–5), and **createdAt**. The API returns the latest reviews (sorted descending, limited) for the home carousel and the admin panel.

12.4 Client-only saved books

Saved titles are kept in the browser under a key per member email (title, author, cover, id). This is intentional in the current version: membership is server-side; the reading list is local to the device.

13. REST API Summary

Method	Path	Auth	Purpose
GET	/	No	Health text
GET	/all-books	No	List catalog
GET	/book/{id}	No	One book
POST	/add-book	Admin UI	Create book
PUT	/update-book/{id}	Admin UI	Update book
DELETE	/delete-book/{id}	Admin UI	Delete book
GET	/all-reviews	No	List reviews
POST	/add-review	JWT required	Create review
POST	/register	No	Create member + JWT
POST	/login	No	Verify password + JWT
GET	/me	JWT required	Current user profile

Table 6. API contract. Interactive documentation is available at <http://127.0.0.1:5000/docs> when the server is running. Book mutation routes are called from the admin dashboard; tightening them with JWT on the server is listed under future work.

14. User Interface Overview

Route	Screen	Access
/	Home (banner, lists, reviews)	Public
/shop	Catalog search and genre filter	Public
/book/:id	Full book + Amazon + save	Public (save needs login)
/about	About the store	Public
/contact	Contact form	Public
/login	Sign in	Public
/sign-up	Create account (then logged in)	Public
/account	Saved books	Logged-in member
/admin/dashboard	Stats overview	Admin only
/admin/dashboard/upload	Add book form	Admin only
/admin/dashboard/manage	Inventory table	Admin only

Route	Screen	Access
/admin/dashboard/edit-books/:id	Edit book	Admin only

Table 7. Client routes (React Router).

15. Hardware and Software Requirements

- OS: Windows 10/11 (project developed on Windows).
- Python 3.13+ with packages in fastapi-server/requirements.txt.
- Node.js and npm for mern-client (npm run dev).
- PostgreSQL 17/18 listening locally (default DATABASE_URL uses port 5432, database bookstore).
- MongoDB on 127.0.0.1:27017, database name Bookinventory.
- Modern browser (Chrome, Edge, Firefox).

Run API: **cd fastapi-server** then **python main.py**. Run UI: **cd mern-client** then **npm run dev**. Client origin is typically <http://localhost:5173/>.

16. Testing Approach

- Manual UI tests: guest browse, member sign-up/login, save book, post review, logout.
- Negative tests: wrong password, duplicate email, admin email blocked on sign-up, non-admin opening /admin/dashboard.
- API checks via FastAPI /docs: register, login, /me, book CRUD, reviews.
- Startup checks: PostgreSQL database auto-created if missing; admin user seeded; optional review seed if the collection is empty.

17. Limitations

- Book create/update/delete endpoints are not yet JWT-enforced on the server (UI is gated).
- Contact form does not persist messages to a database.
- Saved books do not sync across devices.
- No automated unit/e2e test suite in the repository.
- Secrets must be changed before any public deployment (JWT_SECRET, admin password, database URL).

18. Future Work

- Require admin JWT on all book mutation APIs.
- Move saved books into PostgreSQL so lists follow the user.
- Optional: store the full catalog in PostgreSQL for a single database.
- Password reset via email and optional OAuth.
- Server-side contact inbox and order/cart if in-house checkout is required.
- Deploy frontend and API with HTTPS and environment-specific CORS.

19. Project Timeline (Indicative)

The following schedule is suitable for an academic semester mapping onto the modules already present in the codebase.

Week	Milestone
1–2	Requirements, wireframes, stack selection, local Postgres + Mongo setup
3–4	FastAPI book CRUD and React shop/home catalog
5–6	Book detail, Amazon/Goodreads, About/Contact polish
7–8	PostgreSQL auth, JWT, sign up/login, member reviews and saved books
9–10	Admin dashboard, upload/manage/edit, role guard
11–12	Integration testing, proposal/documentation, demo rehearsal

Table 8. Suggested 12-week plan aligned with implemented features.

20. Conclusion

Book Store is a complete student-scale bookstore platform: a React storefront, a FastAPI backend, PostgreSQL for accounts, and MongoDB for books and reviews. It solves incomplete listings and fake checkout by showing full metadata and sending buyers to real external stores, while giving members a reason to register (reviews and a saved list) and giving an administrator a dedicated inventory console. The architecture is ready for classroom demonstration and for incremental hardening (API authorization, synced saved lists, and production deployment) as future work.

21. References and Tools

- FastAPI documentation — <https://fastapi.tiangolo.com/>
- React documentation — <https://react.dev/>
- Vite — <https://vitejs.dev/>
- PostgreSQL — <https://www.postgresql.org/docs/>
- MongoDB Manual — <https://www.mongodb.com/docs/manual/>
- SQLAlchemy 2.0 — <https://docs.sqlalchemy.org/>
- Tailwind CSS — <https://tailwindcss.com/docs>

22. Appendix — Repository Layout

Path	Contents
mern-client/	React SPA (pages, dashboard, auth context, API helper)
mern-client/src/shop/	Shop grid and single-book page
mern-client/src/dashboard/	Admin layout, upload, manage, edit
mern-client/src/api/auth.js	JWT token helper: register, login, /me, authFetch
fastapi-server/main.py	REST routes for books, reviews, auth
fastapi-server/auth.py	bcrypt + JWT helpers and current-user dependency

Path	Contents
fastapi-server/postgres_db.py	SQLAlchemy User model and Postgres startup
fastapi-server/database.py	MongoDB connection and review seed
fastapi-server/models.py	Pydantic request/response models

Table 9. Main folders and files in D:\library.

End of proposal. Fill the title-page names before printing or submitting to a supervisor.